



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Fellisa Anastasya

NIM : 2428240094

Kelas : SI5B

**PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027**

Identitas Praktikum

Nama Mahasiswa	Fellisa Anastasya
NPM	2428240094
Kelas	SI5B
Pertemuan Ke-	5
Topik / Judul Praktikum	RESTful API dengan Express.js: Routing, Middleware, dan CORS
Tanggal Pelaksanaan	24 September 2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1. Memahami konsep dan prinsip RESTful API serta penggunaan metode HTTP (GET, POST, PUT) untuk operasi data.
2. Mampu membangun server web menggunakan Express.js dengan beberapa route, termasuk route dinamis (:id) dan query string.
3. Mampu menerapkan middleware (logger, express.json, dan cors) serta menyimpan konfigurasi dalam file .env menggunakan dotenv.
4. Mampu menguji API dan memahami kebijakan CORS melalui halaman frontend sederhana.

II. Dasar Teori

RESTful API. REST (Representational State Transfer) adalah gaya arsitektur untuk membangun layanan web dengan memperlakukan data sebagai resource yang diakses melalui URL. Pada implementasi proyek ini, operasi resource yang benar-benar tersedia adalah GET untuk membaca, POST untuk menambah, dan PUT untuk memperbarui data. Konfigurasi CORS memang mengizinkan metode DELETE, tetapi route DELETE belum dibuat pada app.js. Respons pada endpoint mahasiswa dikirim dalam format JSON.

Node.js dan Express.js. Node.js digunakan sebagai runtime JavaScript di sisi server, sedangkan Express.js 5.2.1 digunakan sebagai framework web untuk membuat aplikasi server, routing, middleware, dan endpoint API. Dependensi express, cors, dan dotenv tercantum pada package.json, sedangkan nodemon digunakan sebagai dependency pengembangan.

Routing. Routing menentukan respons aplikasi terhadap URL dan metode HTTP tertentu. Pada `app.js` terdapat route dasar `GET /`, `GET /profil`, dan `GET /hubungi`. Untuk resource mahasiswa terdapat `GET /mahasiswa`, `GET /mahasiswa/:id`, `POST /mahasiswa`, dan `PUT /mahasiswa/:id`. Parameter `id` dibaca melalui `req.params`, filter jurusan melalui `req.query`, dan data `POST/PUT` melalui `req.body`.

Middleware. Middleware merupakan fungsi yang diproses di antara request dan response. Proyek menggunakan middleware logger buatan sendiri untuk mencatat waktu, metode, dan URL request; `cors` untuk mengatur origin dan metode HTTP yang diizinkan; serta `express.json()` agar body JSON dapat dibaca oleh endpoint `POST` dan `PUT`. Middleware logger dipasang sebelum route dan memanggil `next()` agar request diteruskan.

CORS dan variabel lingkungan. CORS (Cross-Origin Resource Sharing) dikonfigurasi menggunakan package `cors` dengan origin dari `process.env.CORS_ORIGIN` dan metode `GET`, `POST`, `PUT`, serta `DELETE`. File `.env.example` dalam ZIP berisi `PORT=3000`, `API_KEY`, `CORS_ORIGIN=http://localhost:5173`, dan `DB_URI=mongodb://localhost:27017/paw2`. Namun, screenshot runtime menunjukkan server benar-benar berjalan pada `http://localhost:4000`, sehingga konfigurasi `.env` yang digunakan saat pengujian berbeda dari nilai contoh pada `.env.example`. File `.gitignore` mengabaikan `.env` dan `node_modules/`.

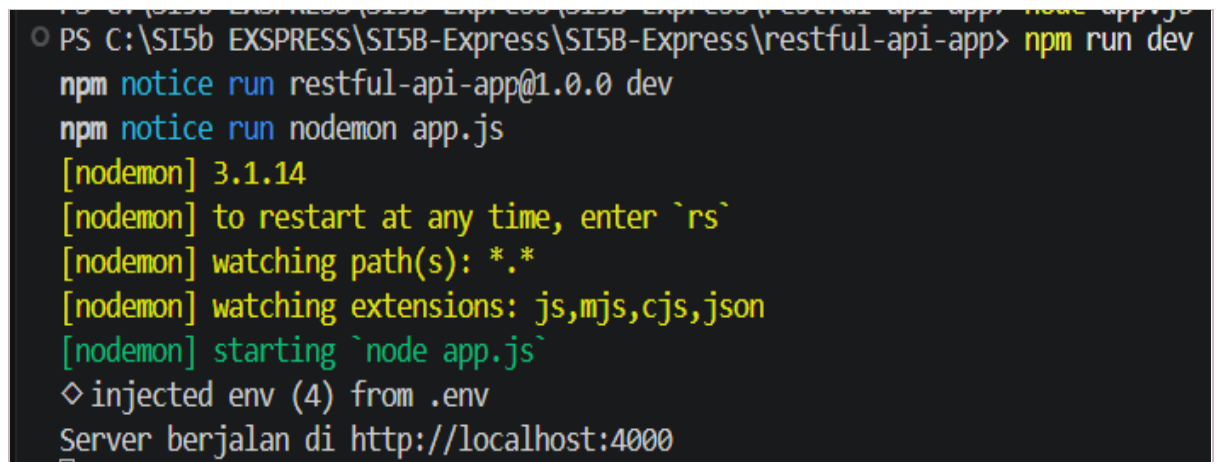
III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	Runtime JavaScript sisi server; versi tidak terlihat pada screenshot dan tidak dicantumkan pada ZIP.
2	Visual Studio Code	Editor kode yang digunakan untuk pengembangan proyek (sesuai konteks laporan/template).
3	Express.js	5.2.1
4	<code>cors</code> , <code>dotenv</code> , <code>nodemon</code>	<code>cors</code> 2.8.6; <code>dotenv</code> 18.0.4; <code>nodemon</code> 3.1.14
5	Postman	Koleksi request mahasiswa tersedia pada folder <code>postman/collections/Express SI5B</code> .
6	Git dan GitHub	Git/GitHub disebut dalam konteks proyek, tetapi URL repository tidak tercantum pada ZIP yang diperiksa.

IV. Langkah Kerja

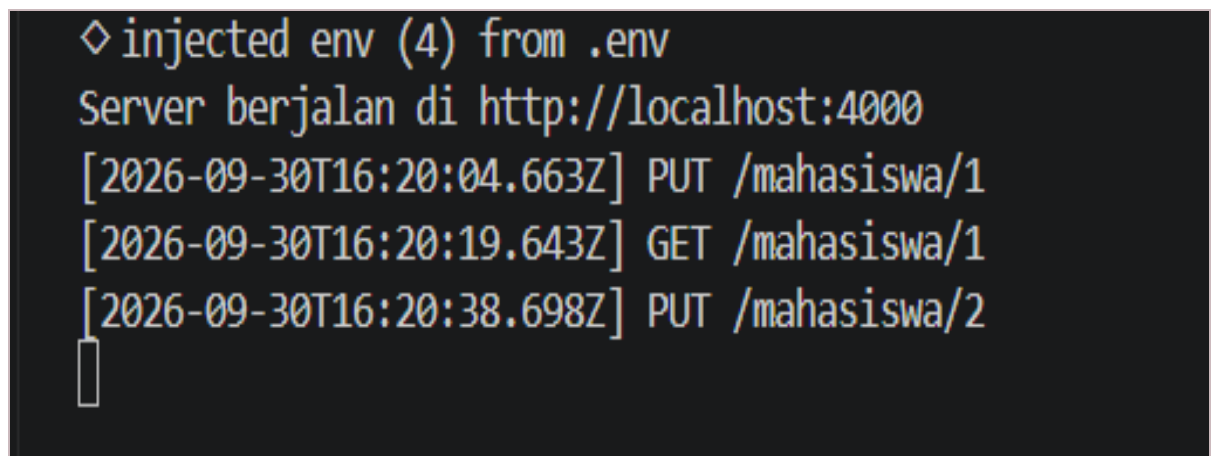
1. Membuat proyek Node.js dan menyiapkan package.json. Package yang digunakan adalah express, cors, dan dotenv sebagai dependencies, sedangkan nodemon sebagai devDependencies. Script yang tersedia adalah start (node app.js) dan dev (nodemon app.js).
2. Menyiapkan konfigurasi melalui .env dan .env.example. File .env.example memuat PORT=3000, API_KEY, CORS_ORIGIN=http://localhost:5173, dan DB_URI=mongodb://localhost:27017/paw2. Namun, pada pengujian yang terlihat pada screenshot, server berjalan pada port 4000, sehingga file .env yang digunakan saat pengujian memiliki nilai PORT=4000. File .gitignore mengabaikan .env dan node_modules/.
3. Membuat app.js dengan memuat dotenv, express, dan cors. Aplikasi Express dibuat dan PORT diambil dari process.env.PORT dengan nilai cadangan 3000 apabila variabel PORT tidak tersedia. Pada pengujian yang terdokumentasi, nilai PORT aktif adalah 4000.
4. Membuat middleware logger yang mencatat waktu, metode HTTP, dan URL setiap request. Logger didaftarkan sebelum route sehingga request yang masuk dapat dicatat sebelum diproses oleh handler.
5. Mendaftarkan middleware cors dengan origin dari CORS_ORIGIN dan metode GET, POST, PUT, DELETE, kemudian menggunakan express.json() agar request body JSON dapat diproses.
6. Membuat route dasar GET / dengan respons 'Server Express.js berjalan!', GET /profil dengan respons 'Ini halaman profil', dan GET /hubungi dengan respons 'Ini halaman hubungi'.
7. Membuat data mahasiswa sementara dalam array memori dengan dua data awal: Andi dari Sistem Informasi dan Budi dari Informatika. Variabel nextId dimulai dari 3 untuk pemberian ID data baru.
8. Membuat endpoint mahasiswa: GET /mahasiswa untuk seluruh data atau filter berdasarkan ?jurusan=; GET /mahasiswa/:id untuk mengambil satu data; POST /mahasiswa untuk menambah data dengan validasi nama dan jurusan; serta PUT /mahasiswa/:id untuk memperbarui data.
9. Menyediakan koleksi request Postman pada folder postman/collections/Express SI5B. Pengujian yang tampak pada screenshot runtime memperlihatkan request PUT /mahasiswa/1, GET /mahasiswa/1, dan PUT /mahasiswa/2. Screenshot yang tersedia tidak menampilkan halaman frontend, sehingga keberhasilan uji CORS melalui browser tidak dinyatakan sebagai hasil yang teramati pada gambar.

V. Hasil dan Pembahasan



```
PS C:\SI5b EXPRESS\SI5B-Express\SI5B-Express\restful-api-app> npm run dev
npm notice run restful-api-app@1.0.0 dev
npm notice run nodemon app.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
◇ injected env (4) from .env
Server berjalan di http://localhost:4000
```

Gambar 1. Server Express berhasil berjalan pada port 4000 menggunakan nodemon (npm run dev), dengan dotenv berhasil memuat 4 variabel dari file .env.



```
◇ injected env (4) from .env
Server berjalan di http://localhost:4000
[2026-09-30T16:20:04.663Z] PUT /mahasiswa/1
[2026-09-30T16:20:19.643Z] GET /mahasiswa/1
[2026-09-30T16:20:38.698Z] PUT /mahasiswa/2
```

Gambar 2. Log middleware logger mencatat request PUT /mahasiswa/1, GET /mahasiswa/1, dan PUT /mahasiswa/2 lengkap dengan timestamp masing-masing request.

Pembahasan:

Pada Gambar 1 terlihat perintah npm run dev menjalankan nodemon versi 3.1.14, kemudian nodemon memulai node app.js. Baris 'injected env (4) from .env' menunjukkan bahwa dotenv memuat 4 variabel lingkungan dari file .env. Baris berikutnya menunjukkan 'Server berjalan di http://localhost:4000', sehingga port runtime yang benar-benar dipakai pada pengujian ini adalah 4000. Hal ini lebih kuat daripada hanya membaca nilai default 3000 pada source code, karena gambar menunjukkan hasil eksekusi aktual.

Pada Gambar 2, middleware logger mencatat tiga request yang benar-benar masuk ke server: PUT /mahasiswa/1 pada 2026-09-30T16:20:04.663Z, GET /mahasiswa/1 pada 2026-09-30T16:20:19.643Z, dan PUT /mahasiswa/2 pada 2026-09-30T16:20:38.698Z.

Urutan tersebut menunjukkan bahwa endpoint PUT dan GET berhasil menerima request pada saat server berjalan. PUT digunakan untuk memperbarui data mahasiswa, sedangkan GET digunakan untuk mengambil data mahasiswa berdasarkan ID. Timestamp pada logger juga menunjukkan bahwa screenshot pengujian direkam pada 30 September 2026, berbeda dari tanggal pelaksanaan yang tercantum pada identitas laporan.

Data mahasiswa pada source code disimpan dalam array di memori, sehingga perubahan yang dilakukan melalui POST atau PUT bersifat sementara dan akan hilang ketika proses server dihentikan atau dimulai ulang. File `.env.example` menyediakan `DB_URI=mongodb://localhost:27017/paw2`, tetapi `app.js` pada versi proyek ini belum menggunakan `DB_URI` untuk koneksi database. Dengan demikian, hasil pengujian yang terdokumentasi masih menggunakan penyimpanan data sementara.

Konfigurasi CORS pada `app.js` membatasi origin berdasarkan `process.env.CORS_ORIGIN` dan mengizinkan GET, POST, PUT, serta DELETE. Namun, screenshot yang tersedia hanya memperlihatkan terminal server dan log request; tidak ada gambar frontend yang menunjukkan request dari origin berbeda. Karena itu, yang dapat dinyatakan secara pasti berdasarkan bukti yang tersedia adalah konfigurasi CORS telah dipasang, sedangkan keberhasilan atau kegagalan uji CORS melalui browser tidak dapat disimpulkan hanya dari screenshot ini.

VI. Kendala dan Solusi

No.	Kendala	Solusi
1	Nilai PORT pada <code>.env.example</code> adalah 3000, sedangkan screenshot runtime menunjukkan server berjalan pada port 4000.	Bedakan nilai contoh dengan konfigurasi yang benar-benar dipakai saat pengujian. Gunakan <code>PORT=4000</code> pada <code>.env</code> untuk mereproduksi hasil screenshot.

VII. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Source code membentuk RESTful API sederhana dengan Express.js menggunakan metode GET, POST, dan PUT untuk resource mahasiswa. Pada pengujian, screenshot menunjukkan server berhasil aktif di `http://localhost:4000` dan middleware logger menerima request dari client.
2. Routing dinamis melalui `/mahasiswa/:id` dan query string melalui `/mahasiswa?jurusan=...` telah diterapkan untuk mengambil data berdasarkan ID dan memfilter data berdasarkan jurusan. Screenshot runtime secara langsung memperlihatkan GET `/mahasiswa/1`.

3. Middleware logger, cors, dan `express.json()` telah diterapkan. Screenshot memperlihatkan logger mencatat waktu, metode, dan URL request, sedangkan dotenv berhasil memuat 4 variabel dari `.env` sebelum server berjalan.
4. Konfigurasi CORS telah dibuat dengan origin dan daftar metode HTTP melalui environment variable. Namun, screenshot yang tersedia tidak menampilkan pengujian frontend lintas origin, sehingga keberhasilan uji CORS tidak dinyatakan sebagai hasil yang terverifikasi dari gambar tersebut.

Daftar Pustaka

Express. (2026). Express - Node.js web application framework. <https://expressjs.com/>

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine].

Mozilla Developer Network. (2026). Cross-Origin Resource Sharing (CORS). MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>

OpenJS Foundation. (2026). Node.js documentation. <https://nodejs.org/docs/latest/api/>

Lampiran

Tautan Repository GitHub:

https://github.com/fellisaanastasya01/si5b_latihan_restful_fellisaanastasya

Tautan Deployment (jika ada):

<https://vercel.com/anastasyafellisa-6661/si5b-latihan-restful-fellisaanastasya>